

Graded Project – DevAnswers (Node & Express)

Introduction

In this project, you will build the backend of **DevAnswers** - a Question & Answer platform for developers. Some scaffolding code (for authentication, tags, error handling, models) is already provided. Your task is to implement the **Question Service**, **Answer Service**, their respective **Controllers**, the **Questions Route**, and the **Tests** that bring the full API to life.

This project will help you practice:

- Implementing service functions for questions and answers — CRUD, voting, and ownership verification
- Building controllers that read from `req` (params, body, user) and respond with the correct HTTP status codes
- Defining Express routes with public and protected (authenticated) endpoints
- Writing unit tests for service functions and integration tests for API endpoints
- Using Github Copilot to help with all the tasks above

Project Setup and Execution

1. Download and extract the project starter code from the zip file provided on Olympus.
2. Navigate to the project folder: `cd devanswers-backend`
3. Install dependencies: `npm install`
4. Create a `.env` file inside `devanswers-backend/` (use `.env.example` as a reference):

```
PORT=3000
NODE_ENV=development
MONGODB_URI=<YOUR_MONGODB_URI> # your MongoDB Atlas connection string
or local MongoDB URI
JWT_SECRET=<YOUR_SECURE_SECRET> # use a long, random string for JWT
secret
JWT_EXPIRATION=7d
```

5. Seed the database and start the server:

```
npm run populate # Seed database with sample data
npm run dev      # Start server with auto-reload (development)
```

Project Structure - Files to Implement

You will have to implement the following in the project:

```
starter_code/
├── .env
├── .github/agents/backend-testing-agent.agent.md
├── AGENTS.md
├── src/
│   ├── controllers/
│   │   ├── answerController.js
│   │   └── questionController.js
│   ├── routes/
│   │   ├── answers.js
│   │   ├── index.js
│   │   └── questions.js
│   ├── services/
│   │   ├── answerService.js
│   │   └── questionService.js
└── tests/
```

Problem Statement

1. Build Custom Instructions

Generate custom instructions for the project (you can use the `/init` command). Carefully review the generated instructions, make any necessary adjustments and save it in `AGENTS.md` in the project root.

2. Implement the Services

1. In Question Service (`src/services/questionService.js`) implement the following functions:

- getAllQuestionsService** - Fetch all the questions in the database. Populate the author name and tags for each question. Attach the `answerCount` to each question, which is the total number of answers for that question. Throw a 404 `AppError` (using the `createAppError` function) if the result is null or empty.
- getQuestionByIdService** - Given the argument `id`, fetch the question by its ID. Increment its `views`, populate the author name and tags. Fetch and attach all answers for the question. Throw a 404 `AppError` (using the `createAppError` function) if the question is not found.
- createQuestionService** - Given the arguments `title`, `description`, `tags`, `author`, create a new question with those field. Accept `tags` as a comma-separated string (e.g. `'javascript, nodejs'`), resolve each tag name to its ID (create the tag if it does not exist), save and return the new question.

d. **updateQuestionService** - Given the arguments `id`, `title`, `description`, `tags` and `loggedInUser`, update the question with the given `id`. Verify that the requester is the owner or an admin, accept `tags` as a comma-separated string, resolve tag names to IDs (create the tag if missing), update the question and return the updated question. Throw a 404 `AppError` if the question is not found, or a 403 `AppError` if the requester is not authorized to update the question.

e. **deleteQuestionService** - Given the arguments `id` and `loggedInUser`, verify that the requester is the owner or an admin, then delete the question with the given `id` and all its answers. Throw a 404 `AppError` if the question is not found, or a 403 `AppError` if the requester is not authorized to delete the question.

f. **upvoteQuestionService / downvoteQuestionService** - Given the arguments `questionId` and `userId`, call the `voteService.handleVote` function with the appropriate arguments. Throw a 400 `AppError` if the vote operation returns a falsy result. Return the updated question document returned by `voteService.handleVote`.

2. In Answer Service (`src/services/answerService.js`) implement the following functions:

a. **getAnswersByQuestionIdService** - Given the argument `questionId`, retrieve all answers for the given `questionId`. Populate the author name of each answer. Throw a 404 `AppError` (using the `createAppError` function) if the list of answers is null or empty.

b. **createAnswerService** - Given the arguments `questionId`, `answerText`, and `author`, create a new answer. Populate the author's name in the newly created answer and return the populated document.

c. **updateAnswerService** - Given the arguments `answerId`, `answerText`, and `loggedInUser`, verify the requester is the owner or an admin, update the answer text, and return the updated document. Throw a 404 `AppError` if the answer is not found, or a 403 `AppError` if the requester is not authorized to update the answer.

d. **deleteAnswerService** - Given the arguments `answerId` and `loggedInUser`, verify the requester is the owner or an admin, then delete the answer. Throw a 404 `AppError` if the answer is not found, or a 403 `AppError` if the requester is not authorized to delete the answer.

e. **upvoteAnswerService / downvoteAnswerService** - Given the arguments `answerId` and `userId`, call the `voteService.handleVote` function with the appropriate arguments. Throw a 400 `AppError` if the vote operation returns a falsy result. Return the updated answer document returned by `voteService.handleVote`.

4. Implement the Controllers

All controller functions are `async`. Error handling is not done inside the controllers — errors thrown by the service layer (e.g. 404, 403, 400 `AppError`) are expected to propagate to a centralized error-handling middleware. Ensure that the response object has the following structure for all successful responses:

```
{
  "success": true,
  "message": "Descriptive success message",
  "data": {} // or [] depending on the endpoint
}
```

1. In Question Controller (`src/controllers/questionController.js`) implement the following functions:
 - a. **getAllQuestions** – Call the service to get all questions.
 - b. **getQuestionById** – Extract `id` from `req.params`. Call the service to get the question by ID.
 - c. **createQuestion** – Extract `{ title, description, tags }` from `req.body` and `author` from `req.user.id`. Call the service to create a question.
 - d. **updateQuestion** – Extract `id` from `req.params` and `{ title, description, tags }` from `req.body`. Call the service to update a question, passing the full `req.user` object so the service can perform ownership/admin verification.
 - e. **deleteQuestion** – Call the delete question service with arguments `req.params.id` and `req.user`, passing the full `req.user` object for ownership/admin verification.
 - f. **upvoteQuestion** and **downvoteQuestion** – Extract the document `id` from `req.params` and `userId` from `req.user.id` (set by the `authenticate` middleware). Call the appropriate service function to upvote/downvote the question.
2. In Answer Controller (`src/controllers/answerController.js`) implement the following functions:
 - a. **getAnswersByQuestionId** – Extract `questionId` from `req.params`. Call the service to get answers for the given question ID.
 - b. **createAnswer** – Extract `questionId` from `req.params`, `answerText` from `req.body`, and `author` from `req.user.id`. Call the service to create an answer. Respond with status **201** (Created) instead of 200 for this endpoint, and include the created answer in the `data` field of the response.
 - c. **updateAnswer** – Extract `answerId` from `req.params` and `answerText` from `req.body`. Call the service to update the answer, passing the full `req.user` object for ownership/admin verification.
 - d. **deleteAnswer** – Extract `answerId` from `req.params`. Call the service to delete the answer, passing the full `req.user` object for ownership/admin verification. No `data` field is included in the response.

e. **upvoteAnswer** and **downvoteAnswer** – Extract **answerId** from **req.params** and **userId** from **req.user.id** (set by the **authenticate** middleware). Call the appropriate service function to upvote/downvote the answer.

5. Create Questions and Answers Routers

a. In **src/routes/questions.js**, define the following endpoints and connect them to the controller functions:

1. Public Routes:

- **GET /** (get all questions),
- **GET /:id** (get question by ID),
- **GET /:questionId/answers** (get answers for a given question)

2. Protected Routes (require **authenticate** middleware):

- **POST /** (create question),
- **PUT /:id** (update question),
- **DELETE /:id** (delete question),
- **POST /:id/upvote** (upvote question),
- **POST /:id/downvote** (downvote question),
- **POST /:questionId/answers** (create answer for a given question)

Export the router and ensure it is mounted in **src/routes/index.js** with the prefix **/questions**.

b. In **src/routes/answers.js**, define the following endpoints and connect them to the Answer Controller functions:

1. Protected Routes (require **authenticate** middleware):

- **PUT /:answerId** (update answer),
- **DELETE /:answerId** (delete answer),
- **POST /:answerId/upvote** (upvote answer),
- **POST /:answerId/downvote** (downvote answer)

Export the router and ensure it is mounted in **src/routes/index.js** with the prefix **/answers**.

6. Create Unit and Integration Tests

Create a custom testing agent and use it to generate unit and integration tests for the Question Service and Questions API endpoints.

- **tests/integration/questions.test.js** - Integration tests for the Questions API endpoints. Generate tests for all endpoints, covering success cases and error cases.
- **tests/unit/services/questionService.test.js** - Unit tests for the Question Service functions. Generate tests for all service functions, covering success cases and error cases.

Generate at least 3 test cases per endpoint/service function. Save your custom agent in `.github/agents/backend-testing-agent.agent.md`.

You can run the tests with the command:

```
npm run test
```

Checklist

Make sure you have implemented the following endpoints. You can use Github Copilot to generate Postman collections to test your API manually.

Method	Endpoint	Description	Auth Required?
GET	/questions	Get all questions with author, tags, and answer count	No
GET	/questions/:id	Get question by ID with author, tags, and all answers	No
POST	/questions	Create a new question with title, description, tags	Yes
PUT	/questions/:id	Update a question (only owner or admin)	Yes
DELETE	/questions/:id	Delete a question (only owner or admin)	Yes
POST	/questions/:id/upvote	Upvote a question	Yes
POST	/questions/:id/downvote	Downvote a question	Yes
GET	/questions/:questionId/answers	Get all answers for a given question	No
POST	/questions/:questionId/answers	Create an answer for a given question	Yes
PUT	/answers/:answerId	Update an answer (only owner or admin)	Yes
DELETE	/answers/:answerId	Delete an answer (only owner or admin)	Yes
POST	/answers/:answerId/upvote	Upvote an answer	Yes
POST	/answers/:answerId/downvote	Downvote an answer	Yes